

Міністерство освіти і науки України
Харківський національний університет радіоелектроніки

Кафедра «Програмна інженерія»

Звіт з лабораторної роботи №2

З дисципліни "Моделювання та аналітика даних"

Проста лінійна регресія (аналітичний розв'язок)

Виконала:
студентка гр. ПЗМ-25-2
Тимощенко Владислава Русланівна

Перевірів: асистент кафедри ПІ
Дашенков Дмитро Сергійович

Харків 2026

1. Мета: ознайомитися з реалізацією простої лінійної регресії методом найменших квадратів за допомогою NumPy та дослідити якість прогнозування на основі різних ознак.

2. Хід роботи

2.1 Реалізація `simple_linear_regression()`

Завершиом функцію, використовуючи лише NumPy (без sklearn, без циклів по рядках).

```
[6] def simple_linear_regression(input_feature, output):  
    """  
    Обчислює slope та intercept МНК для однієї ознаки.  
  
    Параметри  
    -----  
    input_feature : array-like, shape (n,)  
    output        : array-like, shape (n,)  
  
    Повертає  
    -----  
    intercept, slope : float, float  
    """  
    input_feature = np.array(input_feature, dtype=float)  
    output        = np.array(output, dtype=float)  
    n = len(input_feature)  
  
    # Compute slope  
    x_mean = np.mean(input_feature)  
    y_mean = np.mean(output)  
  
    slope = (  
        np.sum(input_feature * output) - n * x_mean * y_mean  
    ) / (  
        np.sum(input_feature ** 2) - n * x_mean ** 2  
    )  
  
    # Compute intercept  
    intercept = y_mean - slope * x_mean  
  
    return intercept, slope  
  
▼ Перевірка — обидва значення нижче мають дорівнювати 1.0  
  
[7] test_feature = np.arange(5, dtype=float)  
test_output = 1.0 + 1.0 * test_feature  
test_intercept, test_slope = simple_linear_regression(test_feature, test_output)  
print(f'Intercept: {test_intercept:.4f} (очікується 1.0)')  
print(f'Slope      : {test_slope:.4f} (очікується 1.0)')  
  
▼ ... Intercept: 1.0000 (очікується 1.0)  
      Slope      : 1.0000 (очікується 1.0)
```

Обчислюються середні значення ознаки та цільової змінної. Далі за формулою методу найменших квадратів визначається коефіцієнт нахилу прямої (slope). Після цього обчислюється вільний член рівняння (intercept). Функція повертає параметри лінійної регресії, отримані без використання бібліотеки scikit-learn.

2.4 Передбачення та RSS

а) Реалізуємо `get_regression_predictions(input_feature, intercept, slope)` — повертає масив NumPy передбачених значень.

```
[9] ✓ Os
def get_regression_predictions(input_feature, intercept, slope):
    """Повертає масив передбачених значень."""

    input_feature = np.array(input_feature, dtype=float)

    predictions = intercept + slope * input_feature

    return predictions
```

У функції `get_regression_predictions()` обчислюються передбачені значення за рівнянням лінійної регресії $y = \text{intercept} + \text{slope} \cdot x$.

б) Реалізуємо `get_residual_sum_of_squares(input_feature, output, intercept, slope)` — обчислює RSS.

```
[10] ✓ Os
def get_residual_sum_of_squares(input_feature, output, intercept, slope):
    """Повертає RSS (скаляр)."""

    predictions = get_regression_predictions(
        input_feature, intercept, slope
    )

    rss = np.sum((output - predictions) ** 2)

    return rss
```

У функції `get_residual_sum_of_squares()` спочатку отримуються передбачення моделі, після чого для кожного об'єкта обчислюється квадрат різниці між фактичним і прогнозованим значенням. Сума всіх квадратів залишків формує показник RSS, який характеризує похибку моделі.

в) Перевіримо обидві функції

```
✓ Перебірка — RSS на тестових вхідних даних має бути 0.0

[11] ✓ Os
rss_check = get_residual_sum_of_squares(
    test_feature, test_output, test_intercept, test_slope
)
print(f'RSS на тестових вхідних даних: {rss_check:.2f} (очікується 0.0)')

... RSS на тестових вхідних даних: 0.00 (очікується 0.0)
```

RSS на навчальній і тестовій вибірках:

```
[12] ✓ Os
    rss_train = get_residual_sum_of_squares(
        train_data['sqft_living'], train_data['price'],
        sqft_intercept, sqft_slope
    )
    rss_test = get_residual_sum_of_squares(
        test_data['sqft_living'], test_data['price'],
        sqft_intercept, sqft_slope
    )
    print(f'Навчальна RSS: {rss_train:.2e}')
    print(f'Тестова RSS: {rss_test:.2e}')
```

Навчальна RSS: 2.74e+14
Тестова RSS: 1.20e+15

Яка передбачувана ціна будинку площею 2 650 кв. футів?

```
[14] ✓ Os
    my_house_sqft = 2650
    # Завдання 3в – обчисліть і виведіть передбачувану ціну

    predicted_price = get_regression_predictions(
        my_house_sqft,
        sqft_intercept,
        sqft_slope
    )

    print(f'Передбачувана ціна: {predicted_price:.2f}')
```

... Передбачувана ціна: 695105.98

Для будинку площею 2650 квадратних футів обчислюється прогнозована ціна за допомогою рівняння побудованої лінійної регресії. Отримане значення є оцінкою вартості будинку на основі ознаки `sqft_living`.

2.5 Порівняння двох ознак

Навчимо другу модель, використовуючи `bedrooms` як вхідну ознаку. Обчислимо RSS на тестовій вибірці для обох моделей (`sqft_living` і `bedrooms`).

Ознака `sqft_living` дає кращий прогноз вартості будинку, оскільки її тестова RSS ($1.20 \cdot 10^{15}$) є меншою за RSS моделі на основі `bedrooms` ($2.15 \cdot 10^{15}$). Це свідчить про те, що житлова площа сильніше пов'язана з ціною будинку, ніж кількість спальень. Тому модель з ознакою `sqft_living` забезпечує точніші прогнози.

2.6 Бонус — Зворотна регресія

Реалізуємо `inverse_regression_predictions(output, intercept, slope)`, яка розв'язує рівняння регресії у зворотному напрямку: за цільовою ціною оцінює необхідну площу `sqft_living`. Побудуємо пряму регресії на діаграмі розсіювання навчальних даних і позначте точку (\$800 000, ваша відповідь) на графіку.

✨ Бонус — Зворотна регресія

Реалізуйте `inverse_regression_predictions(output, intercept, slope)`, яка розв'язує рівняння регресії у зворотному напрямку: за цільовою ціною оцінює необхідну площу `sqft_living`. Побудуйте пряму регресії на діаграмі розсіювання навчальних даних і позначте точку (\$800 000, ваша відповідь) на графіку.

```
[17]
✓ os
def inverse_regression_predictions(output, intercept, slope):
    """Повертає оцінку input_feature для заданого output."""

    output = np.array(output, dtype=float)

    return (output - intercept) / slope

[18]
✓ os
# Бонус — оцініть площу для будинку вартістю $800 000 та побудуйте графік
target_price = 800000

estimated_sqft = inverse_regression_predictions(
    target_price,
    sqft_intercept,
    sqft_slope
)

print(f'Оцінена площа для будинку вартістю $800000: {estimated_sqft:.2f} кв. футів')

# Побудова графіка

plt.figure(figsize=(8, 6))

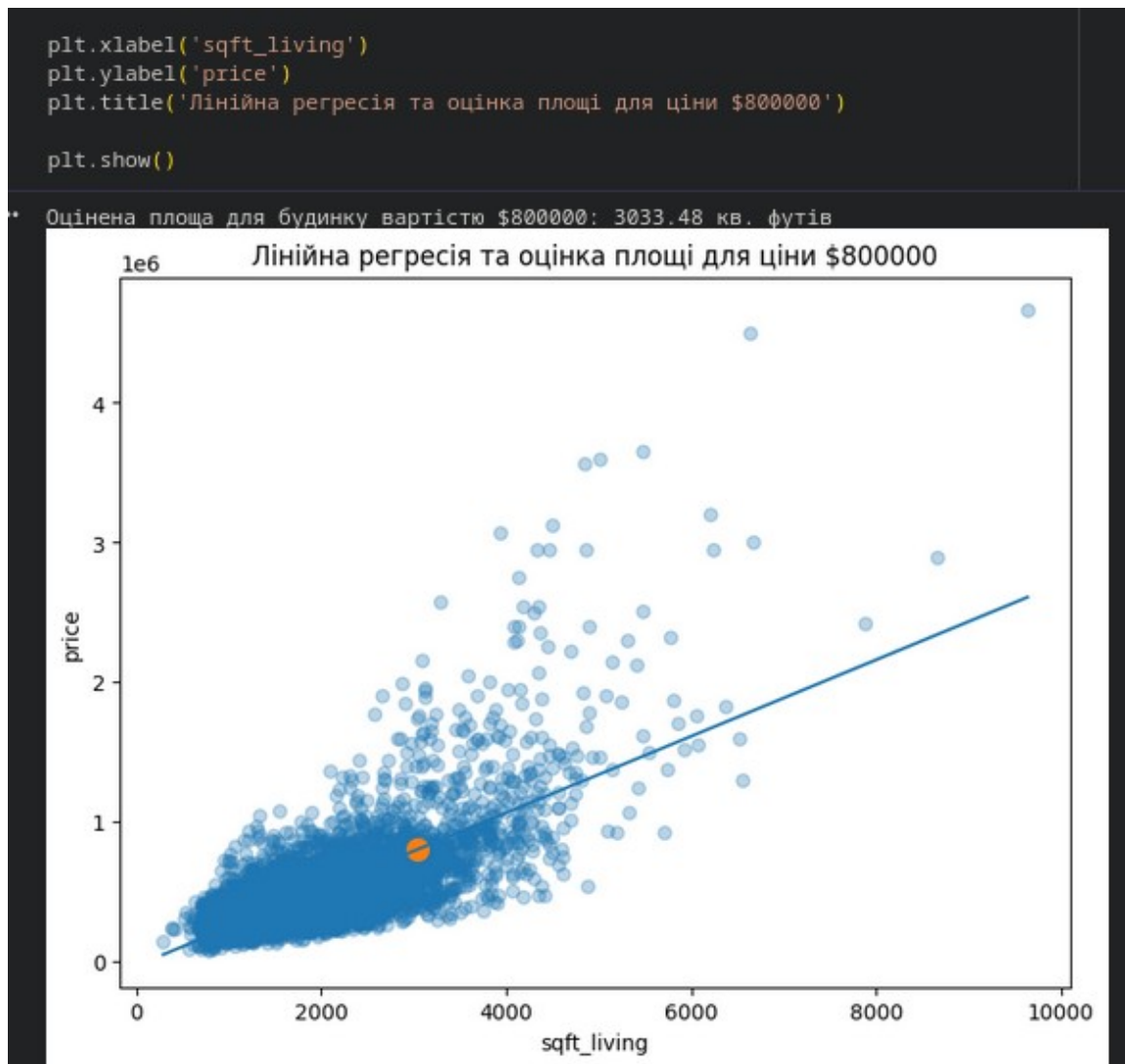
plt.scatter(
    train_data['sqft_living'],
    train_data['price'],
    alpha=0.3
)

x = np.array([
    train_data['sqft_living'].min(),
    train_data['sqft_living'].max()
])

y = sqft_intercept + sqft_slope * x

plt.plot(x, y)

plt.scatter(
    estimated_sqft,
    target_price,
    s=100
)
```



Реалізовується функція зворотної регресії, яка за заданою ціною обчислює відповідне значення ознаки `sqft_living`. Для цього рівняння лінійної регресії розв'язується відносно вхідної ознаки. Далі для ціни 800 000 доларів оцінюється необхідна житлова площа будинку. На діаграмі розсіювання навчальних даних будується пряма регресії та позначається точка, що відповідає отриманій оцінці площі для заданої вартості.

3. Висновки: в ході роботи було реалізовано алгоритм простої лінійної регресії без використання бібліотеки `scikit-learn`, а також функції для прогнозування значень і обчислення RSS. Було побудовано моделі на основі ознак `sqft_living` та `bedrooms` і виконано їх порівняння. Встановлено, що ознака `sqft_living` забезпечує точніший прогноз вартості будинку, оскільки має менше значення RSS на тестовій вибірці.

4. Покликання

Репозиторій GitHub: [покликання](#)

Відео на YouTube: [покликання](#)